

CSCE 640 QUANTUM ALGORITHMS · MODULE 03 · LECTURE NOTES

Gates and Circuits

The operations, and what counts as a cost.

MODULE OUTCOMES

- Use the standard single- and two-qubit gates.
- Explain universality and what it does and does not give.
- Explain reversibility and the cost of uncomputation.
- Build an oracle from a classical function.
- Count the resources of a circuit.

1 The gates

Gate	What it does	What it is used for
X, Y, Z	The Pauli operators: bit flip, bit-and-phase flip, phase flip.	Everywhere — and they are the error basis for Module 10, which is why they matter structurally.
H	Hadamard: maps basis states to even superpositions, and back.	Creating interference, and closing it — almost every algorithm opens and closes with a layer of H.
S, T	Phase rotations by $\pi/2$ and $\pi/4$.	T is the expensive one (Module 10 §4) and the reason T-count is the metric.
$R_z(\theta)$	Arbitrary rotation about z; a relative phase $e^{i\theta}$.	The quantum Fourier transform (Module 07) and variational circuits (Module 11).
CNOT	Flip the target if the control is 1.	The entangling workhorse, and the two-qubit gate most hardware implements natively.
Toffoli (CCNOT)	Flip the target if both controls are 1.	Classical logic done reversibly (§3) — it is reversible AND.

H and CNOT and T is a universal set, and T is by far the most expensive gate to implement fault-tolerantly — it cannot be done transversally in the usual codes and requires magic state distillation — so T-count is the resource metric that actually matters (Module 10 §4). T-count, not gate count, is the number to quote, and a resource estimate that reports only total gates is not telling you the cost.

UNIVERSALITY MEANS APPROXIMATING ANY UNITARY, AND SAYS NOTHING ABOUT EFFICIENCY

A small finite gate set can approximate any unitary on n qubits to any desired accuracy — which is the Solovay-Kitaev result, and it is genuinely reassuring about the model's expressiveness.

And it is entirely silent about cost: almost all unitaries require exponentially many gates, by a straightforward counting argument (there are far more unitaries than short circuits) — **so universality does not mean 'everything is cheap'** and is sometimes read as though it did.

Which is exactly the same structure as classical universality: NAND is universal and most Boolean functions still require exponential-size circuits (CSCE 637 Module 02's counting argument, which transfers unchanged).

So the interesting question is never 'can it be expressed' but 'with how many gates, and at what depth' — and for the overwhelming majority of unitaries the answer is 'far too many', which is why useful quantum algorithms are rare and precious rather than waiting to be found in bulk.

2 Reversibility

EVERY OPERATION EXCEPT MEASUREMENT IS REVERSIBLE, SO NOTHING CAN BE THROWN AWAY

A unitary is invertible, so no information is destroyed by any gate — which means a quantum circuit cannot overwrite a register the way ordinary classical code does, and cannot discard a temporary.

So intermediate results accumulate as garbage in ancillary qubits — and entangled garbage is considerably worse than merely wasteful: it carries which-path information, and therefore destroys the very interference the algorithm needed (Module 04 §3's decoherence argument, in miniature and self-inflicted).

Which forces uncomputation: compute the result, copy the answer out into a fresh register with a CNOT, then run the entire computation backwards to clear the workspace — the Bennett trick — and it roughly doubles the gate count.

And that is the real cost of reversibility: not the restricted gate set, which costs little, but the obligation to clean up — which is the commonest single source of a hand-derived quantum algorithm that does not work, and the first thing to check when a simulation gives the wrong distribution.

3 Oracles

bit oracle: $|x\rangle|y\rangle \rightarrow |x\rangle|y \oplus f(x)\rangle$

phase oracle: $|x\rangle \rightarrow (-1)^{f(x)} |x\rangle$

- The bit oracle is reversible, and it is how any classical function f is made available to a quantum circuit — built from Toffoli, CNOT, and X gates mechanically from any classical circuit for f .
- The phase oracle puts the answer into the phase instead, leaving the input register's basis states alone — which is much cleaner for every interference-based algorithm in this course.
- And the conversion is free: set the target register to $(|0\rangle - |1\rangle)/\sqrt{2}$ before calling the bit oracle, and the target picks up the sign $(-1)^{f(x)}$ while being left unchanged — the phase kickback trick.
- Phase kickback converts a bit oracle into a phase oracle for free, and it is the single most reused trick in the subject: every algorithm from Module 05 onwards uses it, usually without comment.
- Which is worth deriving once by hand, because it then stops being a step to memorise and becomes the obvious thing to do.
- An oracle is a circuit you have to build, not a free lookup table — and its cost counts toward your total, which query-count results deliberately set aside in order to prove clean lower bounds.
- So a query-efficient algorithm is not automatically a time-efficient one — and conflating the two is the commonest error in reading this literature (Module 05 §4 makes this precise).
- And the oracle must be unitary on *all* inputs, including superpositions — which is what 'you may call it but not inspect it' actually means for an implementation: your simulator's oracle must act correctly on a superposed input, which is easy to get wrong by writing it as a classical if-statement.
- Building one from a classical circuit is mechanical — Toffoli for AND, X for NOT, CNOT for fan-out — plus uncomputation of the intermediate wires (§2).
- Which means the oracle usually dominates the gate count of the whole algorithm. A box labelled U_f in a circuit diagram is hiding most of the gate count — which is fine for complexity analysis and actively misleading for a cost estimate.

4 Resources

QUBITS

logical: what the algorithm needs
 physical: logical x overhead (Module 10 s4),
 which is 1000x or more

DEPTH

the longest path; it sets the wall-clock time
 and must fit inside the coherence time (M10)

GATE COUNT

total, and then broken down by type

T-COUNT <-- the one that matters

T gates need magic state distillation, which
 dominates the cost of a fault-tolerant circuit
 Clifford gates (H, S, CNOT, Paulis) are cheap

QUERY COUNT

oracle calls only; a theoretical measure that
 ignores the oracle's own cost (section 3)

AND REPORT ALL FIVE, because a paper that reports
 only the ones that flatter the result is the norm.

AND THE CIRCUIT MODEL IS NOT THE ONLY MODEL, WHICH IS WORTH KNOWING

Measurement-based, adiabatic, and topological models are all polynomially equivalent to the circuit model — so nothing in this course's results depends on the choice, and a result proved in one transfers.

But they differ enormously in engineering difficulty, which is precisely why different hardware efforts pick different ones — and **equivalence in theory is not equivalence in practice**, which is a general lesson this program keeps meeting (CSCE 678 Module 03's consistency models, for one).

And analogue or special-purpose devices are a separate question entirely — a quantum annealer is not a universal quantum computer, and does not run the algorithms in this course — and conflating the two has produced several confusing public claims (Module 13 §3).

So this course uses the circuit model throughout and names it — because 'a quantum computer did X' needs the model stated as part of **Module 13's three requirements**: the problem, the baseline, and the machine.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Nielsen & Chuang, chapter 4 (library copy)	§§1 and 2 — the gate set, universality, and the circuit constructions, with the Solovay-Kitaev discussion.

Resource	What it covers for this module
Bennett — Logical reversibility of computation	§2's uncomputation trick in the original , and it predates quantum computing entirely.
Amy et al. — T-count optimisation (free preprints)	§4's metric — why T-count is the target, and how it is reduced in practice.
The Qiskit transpiler documentation (free)	§4 concretely — run your circuit through it and watch the gate count grow as connectivity constraints are applied.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write the matrices** for X, Z, H, S, T, and CNOT.
- 2 **Verify $H H = I$** and that $H X H = Z$.
- 3 **Count the unitaries versus the short circuits**, and state the consequence.
- 4 **Build a reversible AND** from a Toffoli, and say what the ancilla costs.
- 5 **Compute something, copy it out, and uncompute**, in your simulator.
- 6 **Then skip the uncomputation** and observe the interference fail.
- 7 **Derive phase kickback** by hand.
- 8 **Build a bit oracle** for a three-bit Boolean function, from gates.
- 9 **Count its gates**, and compare to the single box in the diagram.
- 10 **Report all five resource metrics** for one circuit you have built.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name six standard gates and what each is for.
- 2 Give a universal set, and say which gate is expensive and why.
- 3 What does universality give you, and what does it not?
- 4 What is the classical analogue of that distinction?
- 5 Why can nothing be thrown away, and what is entangled garbage?
- 6 Describe uncomputation and its cost.
- 7 Give both oracle forms and the conversion.

- 8 Why is a query-efficient algorithm not necessarily fast?
- 9 Name the five resource metrics, and which dominates.
- 10 Why does the model have to be named in a claim?